

CENTRO UNIVERSITÁRIO UNISENAI
ENGENHARIA MECATRÔNICA

Alexandre Chibeloski Bail

Bruno Martins Guimarães

Diego Lúcio Bojarczuk Fioravanti

Gustavo Machado Aparecido

SISTEMA DE GESTÃO DE EPIs

Documentação Técnica do Código-Fonte

*Trabalho apresentado ao curso de
Engenharia Mecatrônica do Centro
Universitário UNISENAI, como requisito da
disciplina de Design de Software aplicado à
engenharia.*

Curitiba, 14/06/2026

RESUMO

Este documento apresenta a documentação técnica do sistema de gerenciamento de EPIs desenvolvido em linguagem C e Python para a resolução do desafio da jornada de aprendizagem do primeiro período de Engenharia Mecatrônica do UNISENAI. O sistema é composto por três módulos principais: gestão e cadastro de EPIs, gestão e cadastro de colaboradores e controle de validade de EPIs. O código foi desenvolvido para execução em ambiente Windows com o compilador Visual Studio, consistindo de arquivos backend em C, banco de dados em SQL utilizando a biblioteca SQLite, e frontend com GUI em Python utilizando o Tkinter.

Palavras-chave: EPI, Python, Sistema, C, SQLite, Banco de Dados.

SUMARIO

RESUMO	1
SUMARIO	3
1 INTRODUCAO	5
1.1 Contexto e Problema	5
1.2 Objetivos	5
2 TECNOLOGIA UTILIZADA	6
2.1 Linguagem C e SQLite.....	6
2.2 Python e Tkinter	6
2.3 Requests e BeautifulSoup (Web Scraping)	6
2.4 Threading e ThreadPoolExecutor.....	6
3 ESTRUTURA GERAL DO CODIGO	7
3.1 Organizacao por Arquivos	7
3.2 Fluxo de Execucao — Backend C.....	7
3.3 Fluxo de Execucao — Interface Python.....	7
4 ESTRUTURAS DE DADOS.....	9
4.1 Struct EPI (C)	9
4.2 Struct EMPRESTIMO (C)	9
4.3 Validacao de Descricao (Enumeracao Implicita).....	9
4.4 Modelo de Dados no Python	9
4.5 Constante de Alerta de Vencimento	10
5 BANCO DE DADOS SQLITE.....	11
5.1 Tabela epis.....	11
5.2 Tabela emprestimos.....	11
5.3 Operacoes Basicas (CRUD) em Python.....	11
5.4 Consulta com JOIN	12
5.5 Acesso Concorrente entre C e Python.....	12
5.6 Equivalencia em C — sqlite3.h.....	13
6 MODULO DA INTERFACE GRAFICA (PYTHON + TKINTER)	14
6.1 Layout em Abas (Notebook)	14
6.2 Tabela de Estoque com Indicadores de Cor	14
6.3 Cadastro com Validacao Assincrona.....	14
6.4 Regras de Bloqueio no Cadastro	15
6.5 Atualizacao Automatica com Preservacao de Selecao.....	15
6.6 Aba de Alocacoes.....	16
7 MODULO DE VALIDACAO DE CA (WEB SCRAPING).....	17

7.1 Requisicao HTTP	17
7.2 Deteccao de CA Inexistente	17
7.3 Extracao da Data de Validade	18
7.4 Extracao da Situacao do CA.....	18
7.5 Calculo de Dias Restantes	18
7.6 Verificacao Paralela na Inicializacao	19
8 FUNCIONALIDADES DO SISTEMA	20
8.1 Aba Estoque	20
8.2 Aba Alocaoes.....	20
9 TECNOLOGIAS UTILIZADAS.....	20
10 COMO EXECUTAR O SISTEMA	21
10.1 Interface Grafica (Python).....	21
10.2 Backend em C (compilacao)	21
10.3 Geracao de Executavel Standalone (.exe)	21
11 CONSIDERACOES FINAIS.....	22
REFERENCIAS.....	23

1 INTRODUCAO

1.1 Contexto e Problema

A disciplina de Jornada de Aprendizagem do primeiro período do curso de engenharia mecatrônica apresentou o desafio enfrentado pela empresa Guindastes Ribas, que atua no segmento de serviços de içamento, carga, descarga e transporte de máquinas e equipamentos. A empresa em questão apresenta diversos problemas no que diz respeito ao controle do seu estoque de EPIs, para o qual não existe controle rígido de prazos de validade, quantidade em estoque e alocação de EPIs por colaborador.

O sistema apresentado neste documento técnico tem por objetivo sanar esses problemas através da implementação de um banco de dados para os EPIs, registrando o número de C.A, data e status de validade (vencido/válido), e lista de alocações por colaborador.

1.2 Objetivos

O sistema desenvolvido atende aos seguintes objetivos:

- Cadastrar EPIs no estoque, validando o número do CA com uma fonte externa antes de salvar;
- Impedir o cadastro de EPIs com CA inexistente, vencido, ou cuja descrição não corresponda ao tipo de equipamento selecionado;
- Designar EPIs a colaboradores, mantendo o controle de quantidade alocada e disponível;
- Verificar periodicamente a validade de todos os CA's já cadastrados;
- Exibir alertas visuais (em cores) na tabela de estoque conforme o status do CA: válido, vencendo em breve, vencido ou inexistente;
- Manter um histórico de alocações com busca por nome, CA ou descrição, e permitir a devolução de EPIs ao estoque;
- Compartilhar um único banco de dados entre um *backend* em C (console) e uma interface gráfica em Python.

2 TECNOLOGIAS UTILIZADAS

2.1 Linguagem C e SQLite

O backend em C implementa o menu de operações de estoque via terminal: cadastrar, listar, excluir e designar EPIs. Em vez de armazenar os dados em *arrays* na memória, conforme foi elaborado na primeira versão do projeto, o programa utiliza a biblioteca `sqlite3.h` para manter todas as operações diretamente em um arquivo de banco de dados (`epis.db`), o mesmo arquivo utilizado pela interface gráfica em Python. As bibliotecas padrão utilizadas são:

```
1 #include <stdio.h> // entrada/saida: printf, fgets, scanf
2 #include <stdlib.h> // funcoes utilitarias
3 #include <string.h> // strcmp, strtok
4 #include <time.h> // data e hora dos emprestimos
5 #include "sqlite3.h" // API do banco de dados
```

2.2 Python e Tkinter

A interface gráfica foi construída em Python, utilizando a biblioteca Tkinter (nativa da linguagem, sem necessidade de instalação adicional) para a construção de janelas, tabelas, formulários e abas. O Python acessa o mesmo arquivo `epis.db` por meio do módulo `sqlite3`, também nativo da linguagem.

2.3 Requests e BeautifulSoup (Web Scraping)

Para a validação do CA, o sistema realiza requisições HTTP ao site público `consultaca.com` utilizando a biblioteca *requests*, e interpreta o HTML retornado com a biblioteca BeautifulSoup4. Essa combinação permite extrair, do conteúdo textual da página, a data de validade, a situação do CA (valido ou vencido) e a descrição do equipamento, sem depender de uma API oficial.

2.4 Threading e ThreadPoolExecutor

Como as requisições HTTP podem demorar alguns segundos, todas as consultas ao site externo são executadas em threads separadas do *thread* principal da interface gráfica, evitando o congelamento da janela. A verificação de todos os CA's do estoque na inicialização do programa funciona em paralelo com ThreadPoolExecutor, executando múltiplas consultas simultaneamente.

3 ESTRUTURA GERAL DO CODIGO

3.1 Organização por Arquivos

O sistema é dividido em arquivos separados por linguagem e responsabilidade, todos compartilhando o mesmo banco de dados epis.db:

Arquivo	Linguagem	Responsabilidade
main.c	C	Menu principal via terminal (loop de opções)
epi.c	C	Implementação das operações de acesso no banco SQLite
epi.h	C	Declarações de <i>structs</i> , constantes e protótipos de funções
epi_manager.py	Python	Interface gráfica completa, validação de CA e web <i>scraping</i>
epis.db	SQLite	Banco de dados compartilhado, criado automaticamente na primeira execução

3.2 Fluxo de Execução — Backend C

Ao ser iniciado, o programa em C executa a seguinte sequência:

```

1 int main(void) {
2     sqlite3 *db = db_open();
3     db_init(db); // cria as tabelas se nao existirem
4
5     int opcao = 0;
6     while (opcao != 5) {
7         switch (opcao) {
8             case 1: CADASTRAR_EPI(db); break;
9             case 2: MOSTRAR_LISTA(db); break;
10            case 3: EXCLUIR_EPI(db); break;
11            case 4: emprestar(db); break;
12        }
13    }
14    db_close(db);
15 }
```

3.3 Fluxo de Execução — Interface Python

A interface gráfica é construída como uma classe que herda da janela principal do Tkinter. Na inicialização, o banco de dados é preparado, os widgets são construídos, os dados são carregados e o ciclo de atualização automática é iniciado:

```
1 class EPIApp(tk.Tk):
2     def __init__(self):
3         super().__init__()
4         self.title("Gerenciador de EPIs")
5         self.geometry("950x660")
6
7         init_db()          # garante que o banco existe
8         self._setup_styles()
9         self._build_ui()   # constroi widgets e abas
10        self.refresh()      # carrega dados do banco
11        self._auto_refresh() # atualiza a cada 2 segundos
12
13        if SCRAPE_OK:
14            threading.Thread(
15                target=self._check_all_ca_expiry, daemon=True
16            ).start()
```


4 ESTRUTURAS DE DADOS

4.1 Struct EPI (C)

No backend em C, cada EPI é representado pela struct EPI, declarada em epi.h. Embora o armazenamento definitivo seja feito no banco SQLite, essa struct espelha a estrutura das colunas da tabela epis e é usada para passar dados entre funções:

```
1 struct EPI {
2     int id;
3     int CA;
4     int QNTD;
5     int QNTD_ALOCADA;
6     char DESCRICAO[100];
7 };
```

4.2 Struct EMPRÉSTIMO (C)

Representa um registro de designação de EPI a um colaborador, incluindo o horário do empréstimo:

```
1 struct EMPRESTIMO {
2     char nome_funcionario[50];
3     time_t horario_emprestimo;
4     int CA_alocado;
5 };
```

4.3 Validação de Descrição (Enumeração Implícita)

O sistema utiliza um *array* de *strings* válidas constantes para validar os tipos de EPI aceitos. A função filtro_descricao percorre esse array através da função loop “for” e compara com strcmp (da biblioteca <string.h>, retornando 1 (válido) ou 0 (inválido):

```
1 static const char *VALIDOS[] = {
2     "capacete", "luva", "bota", "cinto de seguranca",
3     "mangote", "oculos", "protetor auricular", NULL
4 };
5
6 int filtro_descricao(const char *desc) {
7     for (int i = 0; VALIDOS[i] != NULL; i++)
8         if (strcmp(desc, VALIDOS[i]) == 0) return 1;
9     return 0;
10 }
```

4.4 Modelo de Dados no Python

No Python, os tipos válidos de EPI são definidos na mesma estrutura de lista, compartilhada entre a validação local e a comparação com a descrição retornada pelo site externo:

```

1 VALID_EPIS = [
2     "capacete", "luva", "bota", "cinto de segurança",
3     "mangote", "oculos", "protetor auricular",
4 ]
5
6 # Palavras-chave esperadas na página do CA para cada tipo de EPI
7 DESC_KEYWORDS = {
8     "capacete": ["capacete", "helmet"],
9     "luva": ["luva", "glove"],
10    "bota": ["bota", "calcado", "botina", ...],
11    # ... demais tipos
12 }

```

4.5 Constante de Alerta de Vencimento

Define-se o número de dias antes do vencimento em que o sistema passa a exibir o alerta de “vencendo em breve” (cor amarela na tabela):

```

1 WARN_DAYS = 30 # dias para alerta de vencimento proximo

```

5 BANCO DE DADOS SQLITE

SQLite é um banco de dados relacional que opera a partir de um único arquivo local (epis.db), sem necessidade de servidor. Essa característica o torna ideal para o cenário do projeto: o backend em C e a interface Python são executáveis independentes que precisam compartilhar o mesmo estado, e o fazem lendo e escrevendo no mesmo arquivo de banco de dados.

5.1 Tabela epis

Armazena cada tipo de EPI cadastrado, sua quantidade total, quantidade alocada a colaboradores, e o status de validade do CA obtido na última verificação online:

```
1 CREATE TABLE epis (
2   id      INTEGER PRIMARY KEY AUTOINCREMENT,
3   descricao TEXT NOT NULL,
4   ca      INTEGER NOT NULL,
5   qntd    INTEGER NOT NULL CHECK(qntd >= 0),
6   qntd_alocada INTEGER NOT NULL DEFAULT 0,
7   ca_validade TEXT DEFAULT NULL, -- data de vencimento DD/MM/AAAA
8   ca_status TEXT DEFAULT NULL -- OK | VENCIDO | EXPIRANDO |
INEXISTENTE
9 );
```

5.2 Tabela empréstimos

Registra cada designação de EPI a um colaborador, com o nome do funcionário, o horário do empréstimo e o número do CA alocado. Essa tabela alimenta a aba de alocações da interface gráfica:

```
1 CREATE TABLE emprestimos (
2   id      INTEGER PRIMARY KEY AUTOINCREMENT,
3   nome_funcionario TEXT NOT NULL,
4   horario TEXT NOT NULL,
5   ca_alocado INTEGER NOT NULL
6 );
```

5.3 Operações Básicas (CRUD) em Python

As funções de acesso ao banco são centralizadas em funções auxiliares que abrem uma conexão, executam a operação e retornam o resultado. Exemplo de inserção de um novo EPI:

```
1 def db_insert(desc, ca, qntd):
2     with get_conn() as c:
3         c.execute(
4             "INSERT INTO epis (descricao, ca, qntd, qntd_alocada) "
5             "VALUES (?, ?, ?, 0)",
6             (desc, ca, qntd)
```

```
7 )
```

Exemplo de designação de EPI, que atualiza duas tabelas em uma única transação:

```
1 def db_assign(eid, nome):
2     with get_conn() as c:
3         row = c.execute(
4             "SELECT ca, qntd, qntd_alocada FROM epis WHERE id = ?", (eid,)
5         ).fetchone()
6
7     if row["qntd_alocada"] >= row["qntd"]:
8         return False, "Estoque esgotado."
9
10    c.execute("UPDATE epis SET qntd_alocada = qntd_alocada + 1 WHERE id = ?",
11              (eid,))
12    c.execute(
13        "INSERT INTO emprestimos (nome_funcionario, horario, ca_alocado) "
14        "VALUES (?, ?, ?)", (nome, now, row["ca"])
15    )
```

5.4 Consulta com JOIN

Para exibir a aba de alocações com a descrição do EPI ao lado do nome do colaborador, é necessário relacionar a tabela `emprestimos` com a tabela `epis` pelo número do CA. Utiliza-se um `LEFT JOIN` para que alocações permaneçam visíveis mesmo que o EPI correspondente tenha sido excluído do estoque:

```
1 SELECT emp.nome_funcionario, emp.horario,
2        emp.ca_alocado, epi.descricao
3 FROM emprestimos emp
4 LEFT JOIN epis epi ON epi.ca = emp.ca_alocado
5 ORDER BY emp.id DESC;
```

5.5 Acesso Concorrente entre C e Python

Como dois programas distintos podem acessar o mesmo arquivo de banco de dados, o modo `WAL` (*Write-Ahead Logging*) é ativado. Esse modo permite que operações de leitura não sejam bloqueadas por operações de escrita em andamento, reduzindo a chance de erros de *“database is locked”*:

```
1 def get_conn():
2     conn = sqlite3.connect(DB_PATH)
3     conn.row_factory = sqlite3.Row
4     conn.execute("PRAGMA journal_mode=WAL")
5     conn.execute("PRAGMA foreign_keys=OFF")
6     return conn
```

5.6 Equivalência em C — `sqlite3.h`

No backend em C, a mesma operação de inserção é feita com a API C do SQLite: prepare (compila a instrução SQL), Bind (associa valores aos parâmetros de forma segura, prevenindo SQL Injeto), step (executa) e finalize (libera a memória da instrução):

```
1 const char *sql =
2     "INSERT INTO epis (descricao, ca, qntd, qntd_alocada) "
3     "VALUES (?, ?, ?, 0);";
4
5 sqlite3_stmt *stmt;
6 sqlite3_prepare_v2(db, sql, -1, &stmt, NULL);
7 sqlite3_bind_text(stmt, 1, desc, -1, SQLITE_STATIC);
8 sqlite3_bind_int(stmt, 2, ca);
9 sqlite3_bind_int(stmt, 3, qntd);
10 sqlite3_step(stmt);
11 sqlite3_finalize(stmt);
```

6 MÓDULO DA INTERFACE GRÁFICA (PYTHON + TKINTER)

A interface gráfica substitui o menu de terminal por uma janela desktop, organizada em duas abas: Estoque e Alocações. Foi construída com a biblioteca Tkinter, nativa do Python, dispensando instalação de dependências adicionais para a parte visual.

6.1 Layout em Abas (Notebook)

A janela principal organiza o conteúdo em um widget Notebook, que cria abas navegáveis. A aba Estoque concentra o formulário de cadastro/designação e a tabela de EPIs e a aba alocações apresenta o histórico de designações:

```
1 nb = ttk.Notebook(root)
2 nb.pack(fill="both", expand=True)
3
4 tab_estoque = ttk.Frame(nb, padding=4)
5 nb.add(tab_estoque, text=" Estoque ")
6 self._build_form(tab_estoque) # formulario lateral
7 self._build_table(tab_estoque) # tabela de estoque
8
9 tab_alloc = ttk.Frame(nb, padding=4)
10 nb.add(tab_alloc, text=" Alocações ")
11 self._build_alloc_table(tab_alloc)
```

6.2 Tabela de Estoque com Indicadores de Cor

A tabela (Treeview) recebe tags de cor para cada linha, conforme o status do CA armazenado no banco: vermelho para CA vencido, laranja para CA vencendo em breve, e roxo para CA inexistente. A função `ca_row_tag` decide qual tag aplicar com base na data de validade e no status salvo:

```
1 self.tree.tag_configure("expired", background="#FCEBEB", foreground="#A32D2D")
2 self.tree.tag_configure("expiring", background="#FAEEDA", foreground="#633806")
3 self.tree.tag_configure("invalid", background="#F0E6FF", foreground="#4B0082")
4
5 tag = ca_row_tag(e["ca_validade"], e["ca_status"])
6 self.tree.insert("", "end",
7     iid=str(e["id"]),
8     values=(pos, descricao, ca, qtd, alocados, status, validade),
9     tags=(tag,) if tag else ()
10 )
```

6.3 Cadastro com Validacao Assíncrona

Ao clicar em Cadastrar, o sistema primeiro valida os campos localmente (descricao selecionada, CA e quantidade numéricos e positivos). Em seguida, desabilita o botão e

dispara uma thread em segundo plano para consultar o CA no site externo, evitando que a janela fique congelada durante a requisição:

```

1 def cadastrar(self):
2     # ... validacoes locais dos campos ...
3
4     self.btn_cadastrar.config(state="disabled", text="Verificando CA...")
5     threading.Thread(
6         target=self._check_before_save,
7         args=(desc, ca, qntd), daemon=True
8     ).start()
9
10 def _check_before_save(self, desc, ca, qntd):
11     info = fetch_ca_info(ca) # roda em background
12     # devolve o resultado para a thread principal com segurança
13     self.after(0, lambda: self._on_ca_checked(desc, ca, qntd, info))

```

6.4 Regras de Bloqueio no Cadastro

Após a resposta da consulta, três situações podem bloquear o cadastro, exibindo uma mensagem de erro específica ao usuário:

Situação	Resultado
CA não encontrado na fonte externa	Cadastro bloqueado: "CA nao existe no sistema do MTE"
CA encontrado, porém vencido	Cadastro bloqueado: exibe a data de vencimento
Descrição não corresponde ao CA	Cadastro bloqueado: exibe a descrição real do CA
Sem conexão com a internet	Usuário decide se cadastra mesmo assim (confirmação)

6.5 Atualização Automática com Preservação de Seleção

A tabela é redesenhada automaticamente a cada dois segundos, garantindo que alterações feitas pelo programa em C também apareçam na interface grafica sem necessidade de reiniciar. Para que isso não prejudique a usabilidade, a linha atualmente selecionada é memorizada antes da reconstrução e restaurada em seguida:

```

1 def refresh(self):
2     # salva qual linha estava selecionada
3     sel = self.tree.selection()
4     selected_eid = int(sel[0]) if sel else None
5
6     for row in self.tree.get_children():
7         self.tree.delete(row)
8     # ... insere linhas atualizadas a partir do banco ...
9
10    # restaura a selecao apos reconstruir

```

```

11 if selected_eid and e["id"] == selected_eid:
12     self.tree.selection_set(iid)
13
14 def _auto_refresh(self):
15     self.refresh()
16     self.after(2000, self._auto_refresh) # repete a cada 2s

```

6.6 Aba de Alocações

A aba de Alocações lista o histórico completo de designações, obtido pela consulta com JOIN apresentada na seção 5.4. Um campo de busca filtra os resultados em tempo real, comparando o texto digitado com o nome do colaborador, o número do CA e a descrição do EPI:

```

1 def refresh_alloc(self):
2     query = self.entry_search.get().strip().lower()
3     rows = db_fetch_emprestimos()
4
5     for pos, r in enumerate(rows, start=1):
6         desc = (r["descricao"] or "?").capitalize()
7         nome = r["nome_funcionario"]
8         ca = str(r["ca_alocado"])
9         if query and query not in nome.lower() and \
10            query not in ca and query not in desc.lower():
11             continue
12         self.alloc_tree.insert("", "end", iid=str(r["id"]),
13                                values=(pos, nome, f'CA {ca}', desc, r["horario"]))

```

Também é possível devolver um EPI alugado, o que remove o registro de empréstimo e devolve a unidade ao estoque disponível:

```

1 def db_delete_emprestimo(emp_id, ca):
2     with get_conn() as c:
3         c.execute("DELETE FROM empréstimos WHERE id = ?", (emp_id,))
4         c.execute(
5             "UPDATE epis SET qntd_alocada = MAX(0, qntd_alocada - 1) "
6             "WHERE ca = ?", (ca,)
7         )

```


7 MÓDULO DE VALIDACAO DE CA (WEB SCRAPING)

Este é o módulo responsável por garantir que apenas EPIs com CA válido e compatível com a descrição informada entrem no sistema. A validação é feita consultando o site público “consultaca.com”, que disponibiliza, para cada número de CA, a situação (válido ou vencido), a data de validade e a descrição do equipamento.

7.1 Requisição HTTP

A biblioteca requests realiza a requisição HTTP para a URL construída com o número do CA. Um cabeçalho *User-Agent* de navegador é enviado para que o servidor responda normalmente:

```

1 HEADERS = {
2     "User-Agent": (
3         "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
4         "AppleWebKit/537.36 (KHTML, like Gecko) "
5         "Chrome/124.0.0.0 Safari/537.36"
6     ),
7 }
8
9 resp = requests.get(
10     f"https://consultaca.com/{ca_number}",
11     headers=HEADERS, timeout=12
12 )

```

7.2 Detecção de CA Inexistente

Quando um número de CA não existe na base do Ministério do Trabalho, o site consultaca.com retorna a própria página inicial, que contém produtos e números de CA aleatórios de outros equipamentos. Comparar apenas se o número aparece em algum lugar do texto geraria falsos positivos.

A solução adotada explora um padrão consistente da página de um CA válido: o número aparece isolado, em uma linha própria, imediatamente após o rotulo N CA:. Esse padrão nunca ocorre na página inicial:

```

1 # Pagina de CA valido:  Pagina de CA invalido:
2 # "N CA:"              (padrao nao existe)
3 # "38664"
4
5 lines = full_text.splitlines()
6 ca_found = False
7 for i, line in enumerate(lines):
8     if re.search(r"n[o]?s*ca\s*:\s*$", line.strip().lower()):
9         for j in range(i + 1, min(i + 4, len(lines))):
10             if lines[j].strip() == ca_str:
11                 ca_found = True

```

```

12         break
13
14 if not ca_found:
15     return {"found": False, "error": f"CA {ca_number} nao encontrado."}

```

7.3 Extração da Data de Validade

O site separa, em linhas de texto distintas, o rótulo validade e o valor correspondente. Além disso, o valor vem junto com um texto descritivo (por exemplo, 29/04/2026 venceu há 41 dias). Um padrão de expressão regular extrai apenas os 10 caracteres no formato DD/MM/AAAA, ignorando o restante:

```

1 # Pagina retorna:
2 # "Validade:"
3 # "29/04/2026venceu ha 41 dias" <- data colada com texto
4
5 for i, line in enumerate(lines):
6     if "validade" in line.strip().lower():
7         m = re.search(r"\d{2}/\d{2}/\d{4}", line)
8         if m:
9             result["validade"] = m.group() # "29/04/2026"
10        else:
11            # procura nas proximas linhas
12            for j in range(i + 1, min(i + 4, len(lines))):
13                m = re.search(r"\d{2}/\d{2}/\d{4}", lines[j])
14                if m:
15                    result["validade"] = m.group()
16                break

```

7.4 Extração da Situação do CA

O mesmo padrão de rótulo em uma linha e valor na linha seguinte se aplica ao campo Situação, que pode conter os valores VÁLIDO ou VENCIDO. Quando a situação informada é VENCIDO, o sistema prioriza essa informação sobre o cálculo manual de dias restantes:

```

1 if not result["situacao"] and "situa" in low:
2     if "vencido" in low:
3         result["situacao"] = "VENCIDO"
4     elif "v" in low and "lido" in low:
5         result["situacao"] = "VALIDO"
6     else:
7         for j in range(i + 1, min(i + 4, len(lines))):
8             nxt = lines[j].strip().lower()
9             if "vencido" in nxt:
10                result["situacao"] = "VENCIDO"
11            break

```

7.5 Calculo de Dias Restantes

A função `check_validity` converte a data de validade extraída em um objeto de data e calcula a diferença em dias para a data atual. O resultado é um par (válido, dias), onde válido é uma variável booleana e dias pode ser positivo (CA ainda válido) ou negativo (CA vencido):

```
1 def check_validity(validade_str):
2     try:
3         exp = datetime.strptime(validade_str, "%d/%m/%Y")
4         days = (exp - datetime.today()).days
5         return days >= 0, days
6     except ValueError:
7         return None, None
```

7.6 Verificação Paralela na Inicialização

Ao abrir o sistema, todos os EPIs já cadastrados tem seu CA verificado novamente, para detectar vencimentos ocorridos desde o último uso do programa. Para não tornar a inicialização lenta, as consultas são feitas em paralelo com `ThreadPoolExecutor`, limitando-se a 4 requisições simultâneas:

```
1 with ThreadPoolExecutor(max_workers=4) as pool:
2     futures = {pool.submit(check_one, e): e for e in epis}
3     for future in as_completed(futures, timeout=60):
4         e, info = future.result()
5         # define status: OK, VENCIDO, EXPIRANDO ou INEXISTENTE
6         db_update_ca_status(e["id"], validade, status)
```

Caso algum EPI seja identificado como vencido, vencendo em breve, ou com CA inexistente, uma janela de alerta é exibida ao usuário logo após a abertura do sistema, listando todos os itens com problemas e o número de dias restantes ou de atraso.

8 FUNCIONALIDADES DO SISTEMA

8.1 Aba Estoque

Funcionalidade	Descricao
Cadastrar EPI	Valida CA online antes de salvar no banco de dados
Designar EPI	Atribui um EPI a um colaborador, com registro de data e hora
Excluir EPI	Remove total ou parcialmente unidades do estoque
Indicadores de cor	Vermelho = vencido, laranja = vence em breve, roxo = CA inexistente
Atualização automática	Tabela atualizada a cada 2 segundos
Alerta na inicialização	Janela de aviso para CAs vencidos, vencendo ou inexistentes

8.2 Aba Alocações

Funcionalidade	Descricao
Histórico completo	Lista todas as designações com data, hora, CA e descricao
Busca em tempo real	Filtra por nome do colaborador, número de CA ou descricao
Devolução de EPI	Remove a alocação e devolve a unidade ao estoque disponível

9 TECNOLOGIAS UTILIZADAS

Tecnologia	Versão	Uso no projeto
C (GCC/MSVC)	C11	Backend de console e operações básicas no banco SQLite
SQLite	3.x	Banco de dados local compartilhado entre C e Python
Python	3.12	Interface grafica e modulo de validacao de CA
Tkinter	Nativa	Construção de janelas, tabelas, abas e formulários
requests	2.x	Requisições HTTP ao site consultaca.com
BeautifulSoup4	4.x	Interpretação do HTML retornado pelo site
threading / ThreadPoolExecutor	Nativa	Execução assíncrona e paralela das consultas

Tecnologia	Versão	Uso no projeto
PyInstaller	6.x	Geração de executável <i>standalone</i> para Windows

10 COMO EXECUTAR O SISTEMA

10.1 Interface Grafica (Python)

```
1 pip install requests beautifulsoup4
2 python epi_manager.py
```

10.2 Backend em C (compilação)

```
1 gcc main.c epi.c sqlite3.c -o programa.exe -I. -lpthread
2 .\programa.exe
```

10.3 Geração de Executável Standalone (.exe)

Para distribuir a interface grafica sem que o usuário final precise instalar Python, utiliza-se a ferramenta PyInstaller, que empacota o interpretador, as bibliotecas e o código em um único arquivo executável:

```
1 pip install pyinstaller
2 pyinstaller --onefile --windowed --name GerenciadorEPIS epi_manager.py
3 # Executavel gerado em: dist/GerenciadorEPIS.exe
```

11 CONSIDERAÇÕES FINAIS

O Sistema de Gestão de EPIs desenvolvido atende ao objetivo de automatizar o controle de Equipamentos de Proteção Individual, combinando três tecnologias complementares em uma arquitetura integrada por meio de um banco de dados compartilhado. As principais decisões técnicas adotadas foram:

- Banco de dados único (epis.db): permite que o backend em C e a interface em Python operem de forma independente, mas sempre sincronizada, sem necessidade de um servidor de banco de dados;
- Validação online no momento do cadastro: impede que EPIs com CA inválido, vencido ou de descrição incompatível entrem no sistema, reduzindo erros de digitação e não-conformidades;
- Detecção de CA inexistente por padrão estrutural: em vez de depender de uma API oficial, o sistema identifica corretamente um CA inexistente analisando o padrão de exibição da página, mesmo quando o site retorna a página inicial com dados de outros produtos;
- Operações assíncronas: o uso de threads garante que a interface gráfica permaneça responsiva durante as consultas à internet, que podem levar alguns segundos;
- Verificação periódica de vencimento: ao abrir o sistema, todos os CAs em estoque são verificados novamente em paralelo, e o usuário é alertado sobre quaisquer pendências antes mesmo de iniciar o uso do sistema;
- Indicadores visuais por cor: a tabela de estoque comunica o status de cada CA de forma imediata, sem necessidade de abrir detalhes de cada item;
- Distribuição simplificada: a geração de um executável standalone com PyInstaller permite que o sistema seja utilizado em qualquer computador com Windows, sem necessidade de instalar Python ou bibliotecas.

Como evolução futura, sugere-se a implementação de relatórios exportáveis em PDF, um módulo de notificações por e-mail para CAs próximos do vencimento, e a migração do banco de dados para um modelo cliente-servidor (como PostgreSQL) caso o sistema precise ser utilizado por múltiplos usuários simultaneamente em rede.

REFERENCIAS

DEITEL, H. M.; DEITEL, P. J. C: Como Programar. 6. ed. Sao Paulo: Pearson Prentice Hall, 2011.

KERNIGHAN, B. W.; RITCHIE, D. M. A Linguagem de Programação C. 2. ed. Rio de Janeiro: Campus, 1989.

SQLITE CONSORTIUM. *SQLite Documentation*. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 14 jun. 2026.

PYTHON SOFTWARE FOUNDATION. tkinter — Python interface to Tcl/Tk. Disponível em: <https://docs.python.org/3/library/tkinter.html>. Acesso em: 14 jun. 2026.

REQUESTS. Requests: HTTP for Humans. Disponível em: <https://requests.readthedocs.io>. Acesso em: 14 jun. 2026.

RICHARDSON, L. Beautiful Soup Documentation. Disponível em: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Acesso em: 14 jun. 2026.

CONSULTACA. Portal de Consulta de Certificado de Aprovação (CA) de EPIs. Disponível em: <https://consultaca.com>. Acesso em: 14 jun. 2026.

ASSOCIACAO BRASILEIRA DE NORMAS TECNICAS. NBR 6023: informação e documentação — referencias — elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIACAO BRASILEIRA DE NORMAS TECNICAS. NBR 6024: informação e documentação — numeração progressiva das seções de um documento — procedimento. Rio de Janeiro: ABNT, 2012.